iTestStudio TA

Bedienungsanleitung der Testautomatisierung
iTestStudio TA 5.2.0

Version	Datum	Bearbeiter	Anmerkungen/Änderungen
1.0.0	10.10.2014	Tremmel	angelegt
1.1.0	16.10.2014	Trenkel	Ergänzungen
1.2.0	20.10.2014	Tremmel	Installationsanleitung ergänzt
1.3.0	20.10.2014	Felkl	Review, Ergänzungen zur Installation
1.4.0	06.08.2018	Tremmel	Kommandozeilenoptionen ergänzt, Screenshots aktualisiert. Kapitel zu Testreports und Konfigurationseinstellungen überarbeitet.
1.5.0	07.08.2018	Tremmel	Exit-Statuswerte und Kommandozeilenbeispiel ergänzt
1.6.0	03.03.2020	Tremmel	Systemumgebung übernommen, Screenshots aktualisiert, Kapitel „Testprojekt wählen“ ergänzt
1.7.0	08.09.2020	Tremmel	Python 3.8 ergänzt, Test-Reports-Screenshot aktualisiert
1.8.0	06.07.2021	Tremmel	Kapitel „Ablagestruktur für Testergebnisse“
1.8.1	30.11.2022	Tremmel	Beschreibungen der Kommandozeilenoptionen erweitert
1.8.1	07.07.2023	Tremmel	Version für Patch-Release 5.1.2 angepasst
1.9.0	05.04.2024	Tremmel	Überarbeitet für iTS-TA Version 5.2.0

INHALT

1 iTestStudio-Komponenten ... 4
2 Installation der Testautomatisierung ... 5
2.1 Systemumgebung ... 5
2.2 Installation ... 5
3 iTestStudio TA ... 7
3.1 Testprojekt wählen ... 8
3.2 Teststruktur bearbeiten ... 9
3.2.1 Testserien bearbeiten ... 10
3.2.2 Testsequenzen bearbeiten ... 12
3.3 Testdurchführung starten ... 14
3.4 Testreports erstellen ... 17
3.4.1 Testreports für den aktuellen Testlauf ... 17
3.4.2 Testreports zu bereits bestehenden XML-Reports ... 19
3.4.3 Vergleichen von Testreports ... 20
4 Anhang ... 21
4.1 Kommandozeilenoptionen ... 21
4.1.1 Rückgabewerte ... 22
4.1.2 Beispiel ... 23
4.2 Programmkonfiguration ... 24
4.3 Projektkonfiguration ... 24
4.3.1 Beispiel: venv für ein HIL-Projekt konfigurieren ... 26
4.3.2 Beispiel: Einen abweichenden Pythoninterpreter konfigurieren ... 26
4.3.3 Beispiel: TestHandler für ein Projekt definieren ... 27
4.4 Standard-HIL-Projektstruktur ... 28
4.5 Testskripte im iTestStudio ... 29
4.5.1 Schnittstelle ... 29
4.5.2 Ergebnisformat ... 29
4.5.3 Beispielskript mit TestEnv ... 30
4.5.4 Beispielskript ohne TestEnv ... 32
4.6 Test-Handler-Skripte im iTestStudio ... 34

4.6.1 Schnittstelle 34
4.6.2 Beispielskript 34
4.7 Ablagestruktur für Testergebnisse 36
4.7.1 Klassische Struktur 36
4.7.2 Container-Struktur 36
4.7.3 Konfiguration 37
4.8 Aufbau der Lizenzdatei 37
4.9 Abbildungsverzeichnis 38

1 iTestStudio-Komponenten

Die Testsuite „iTestStudio“ unterteilt sich im Wesentlichen in drei Komponenten:

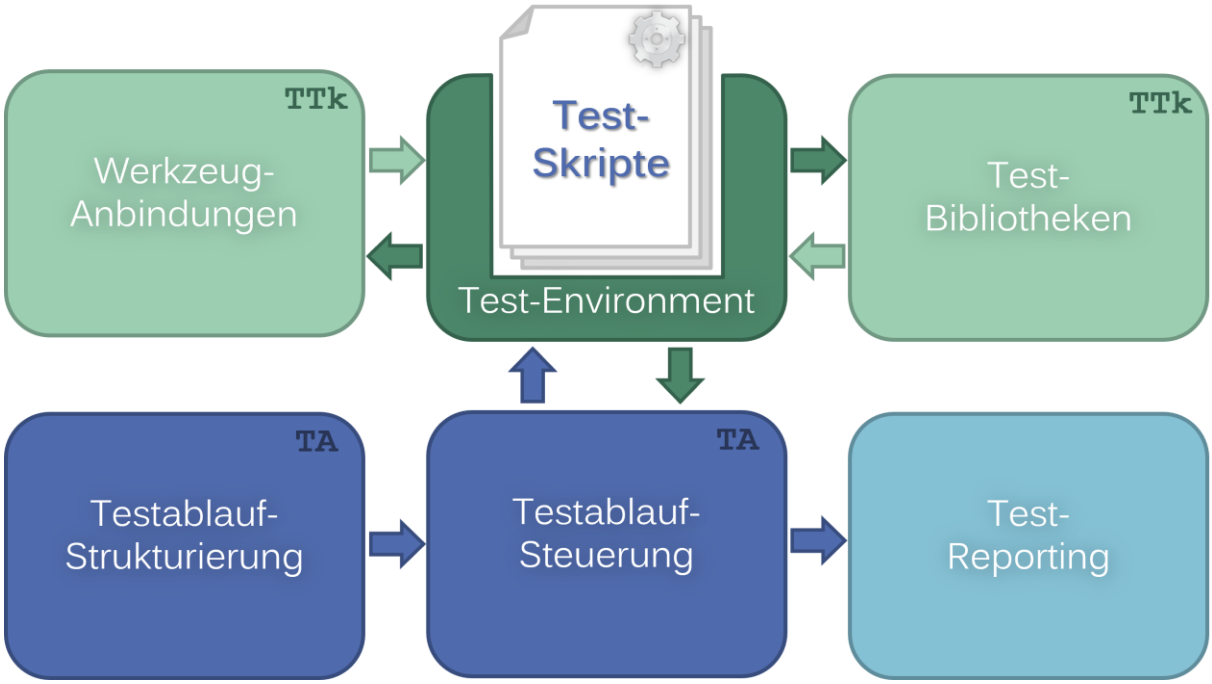


Abbildung 1: Komponenten im iTestStudio

- iTestStudio TA – Benutzeroberfläche zur Testautomatisierung
  - Zusammenstellung der Teststruktur
  - Testablaufsteuerung
  - Testreporting

- TestSupport-Bibliotheken – Test Environment
  - Basis für Testumgebung
  - Projektübergreifende Hilfsbibliotheken

- TestToolkit-Bibliotheken – Test Libraries
  - Bibliotheken zur Testerstellung

In dieser Bedienungsanleitung wird im Weiteren auf die grafische Benutzeroberfläche iTestStudio TA (Test Automation) eingegangen.

2 Installation der Testautomatisierung

2.1 Systemumgebung

- Windows 10 x64
- Unterstützte Python-Versionen für Testskripte:
  - Python 2.7 (x86 oder x64)
  - Python 3.6, 3.8, 3.9, 3.10, 3.11, 3.12 (jeweils x64)
- Microsoft Visual C++ 2017 Redistributables, x86 und x64
- USB-Port

2.2 Installation

Hinweis: Für die Installation sind lokale Administratorrechte erforderlich.

Starten Sie den Installationsassistenten und folgen Sie den Anweisungen am Bildschirm. Der Dongle sollte vor der Erstinstallation noch nicht gesteckt sein.

Es empfiehlt sich, als Installationsort ein Verzeichnis zu wählen, auf das später auch ohne lokale Administratorrechte zugegriffen werden kann, um das spätere Aktualisieren von Lizenzinformationen zu vereinfachen.

Der empfohlene Standard-Installationsort ist: **D:\Tools\iTestStudio**.

Während des Installationsverlaufs wird der Treiber für den Dongle registriert und eine Umgebungsvariable **ITESTSTUDIO_PATH** angelegt, die auf das verwendete Installationsverzeichnis verweist. Hierüber können später aus HIL-Testprojekten heraus die TestSupport-Bibliotheken und Lizenzinformationen lokalisiert werden.

Haben Sie eine Lizenzdatei („**license.its**“) erhalten, so kopieren Sie sie bitte ins Installationsverzeichnis, haben Sie dagegen den Lizenzschlüssel in Textform erhalten, so fügen Sie den Schlüssel bitte am Ende der bestehenden Lizenzdatei ein. Details hierzu finden sie im Anhang im Abschnitt „Aufbau der Lizenzdatei“.

Inhalt des iTestStudio-TA-Installationsordners mit Position der Lizenzdatei:

```
iTestStudio/
    doc/
    libraries/
    resources/
    settings/
    dongle.dll
    dongle_x64.dll
    iTestStudioTA.exe
    license.its          ←    Lizenzdatei
```

Hinweis: Wenn Sie Python zur Testautomatisierung verwenden wollen, stellen Sie bitte zudem sicher, dass ein geeigneter Python-Interpreter installiert und im Systempfad eingetragen ist (Umgebungsvariable **PATH**). Aktuelle Python-Versionen bieten dazu auch schon eine Option während der Installation.

Je nach Rechten des aktiven Benutzers empfiehlt sich nach der Installation ein Neustart des Rechners, um alle vorgenommenen Einstellungen sicher zu übernehmen.

Jetzt kann auch der Dongle in einen freien USB-Port eingesteckt werden.

3 iTestStudio TA

Nach dem Programmstart erscheint das Hauptfenster, das neben der Wahl des aktiven HIL-Projektordners und der aktiven Testserie direkten Zugriff auf Aktionen des Testablaufs in drei Hauptbereichen bietet:

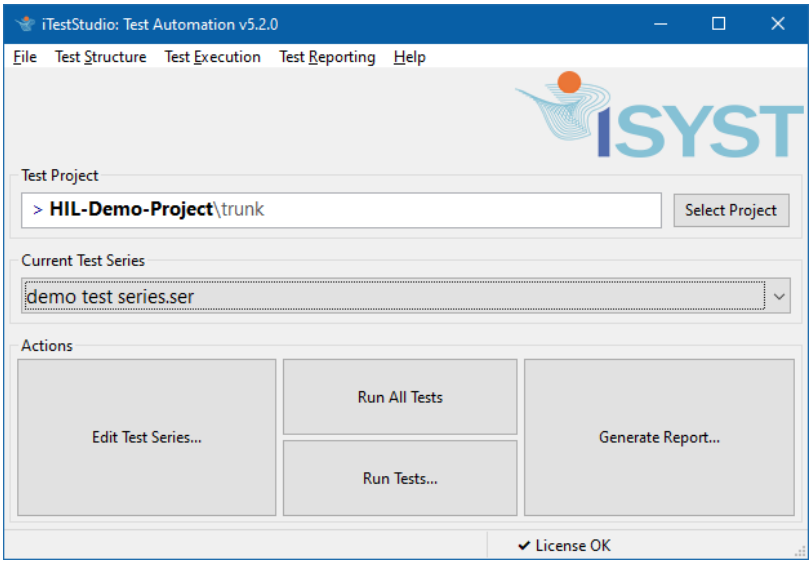


Abbildung 2: Hauptfenster

Die drei Hauptbereiche:

- **Teststruktur bearbeiten** (Testserie bearbeiten... / Edit Test Series...)
  Eine Testablaufstruktur über Testserien und Testsequenzen aus einzelnen Testskripten erstellen oder bestehende Strukturen bearbeiten.

- **Testdurchführung starten** (Tests starten... / Run Tests...)
  Alle Tests der Testserie automatisiert durchführen oder Nachtests über eine Vorauswahl einzelner Tests durchführen

- **Testreports erstellen** (Testreport erstellen... / Generate Report ...)
  Die Einzelergebnisse eines durchgeführten Testlaufs zu einem Gesamtergebnis zusammenfassen und daraus einen Testbericht erstellen

3.1 Testprojekt wählen

Über den Button **„Select Project“** („Projekt auswählen“) oder den Menüpunkt **„File“ → „Select Project ...“** („Datei“ → „Projekt auswählen...“) kann ein aktiver Projektordner gewählt werden.

Im Projektordner wird eine vorgegebene Struktur mit Unterordnern erwartet, deren Namen und Positionen jedoch konfigurierbar sind. Siehe dazu die Kapitel 4.2 „Projektkonfiguration“ und 4.4 „Standard-HIL-Projektstruktur“.

Eine Basisprojektstruktur:

```
Projektordner/
    hil_project.ini                 # Projektkonfigurationsdatei
    Python/
        TestLibraries/              # projektspezifische Bibliotheken
        TestPool/                   # Testskripte
    TestReports/                    # Ablageort für Testberichte
        SingleTestResults/          # Speicherort für Einzeltestergebnisse
    TestStructure/                  # Teststrukturdaten; Testserien (.ser)
        TestSequences/              # ... und Testsequenzen (.seq)
```

Um den Menüpunkt **„File“ → „New Project ...“** („Datei“ → „Neues Projekt...“) kann aber auch einfach ein neuer Standard-Projektordner inklusive der benötigten Projektstruktur angelegt werden. Wird „Neues Projekt...“ auf einen bereits bestehenden Projektordner angewendet, so wird die vorliegende Projektstruktur überprüft und gegebenenfalls fehlende Ordner angelegt.

3.2 Teststruktur bearbeiten

Die grundlegende Struktur von HIL-Tests mit dem iTestStudio besteht aus einer Aufteilung in drei Schichten, wie dies in Abbildung 3 dargestellt ist. Ein Testablauf wird in einer Testserie angelegt, in der eine Abfolge von Testsequenzen definiert ist, in denen wiederum die einzelnen Testskripte festgelegt werden.

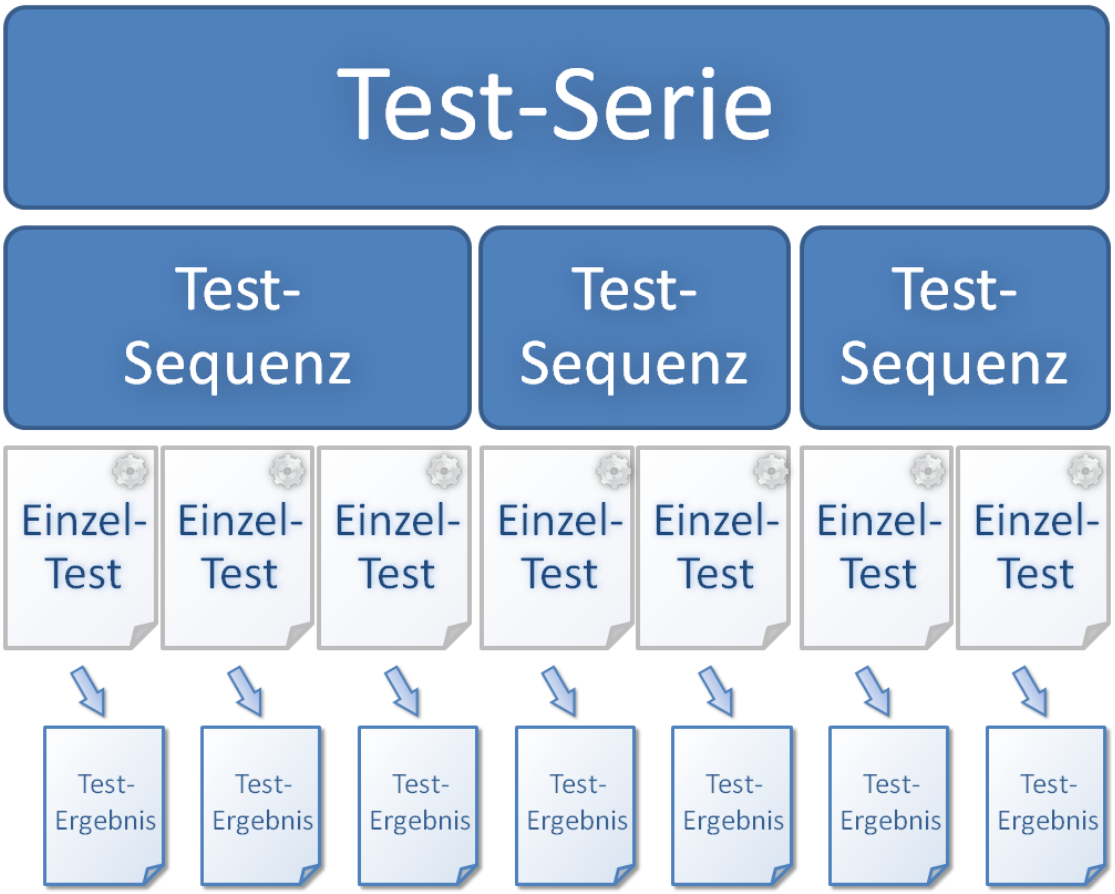


Abbildung 3: Teststruktur

Jeder in Python implementierte Einzeltest wird im Testablauf ein eigenständiges Testergebnis erzeugen. Im abschließenden Schritt der Testberichterstellung werden die Testablaufstruktur und alle dazu vorliegenden Einzeltestergebnisse zu einem Gesamtergebnis aggregiert.

3.2.1 Testserien bearbeiten

Über den Button **„Edit Test Series“** („Testserie bearbeiten“) im Hauptfenster wird der Dialog zum Bearbeiten von Testserien aufgerufen, welcher in Abbildung 4 zu sehen ist.

Ist bereits eine Testserie aktiv, so wird sie automatisch geladen, andernfalls kann über **„Load Test Series“** („Testserie laden“) eine bestehende Testserie geöffnet werden.

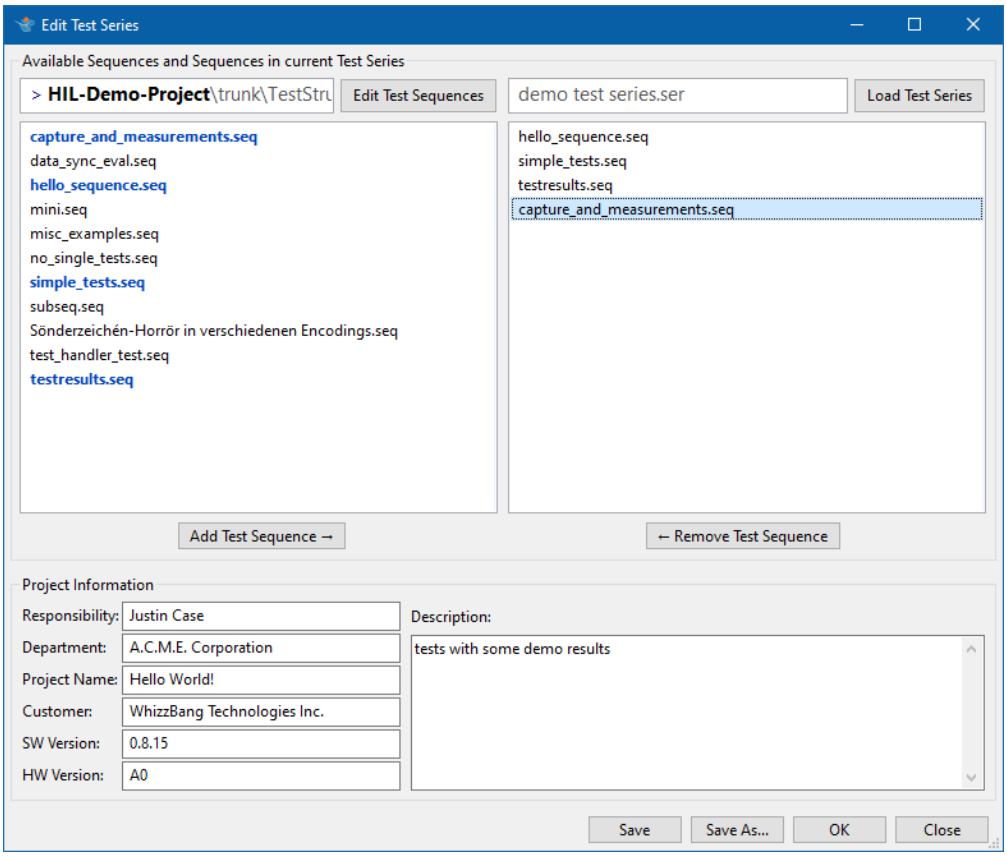


Abbildung 4: Testserie bearbeiten

Verfügbare Testsequenzen des aktuellen Projekts werden im linken Listenfeld angezeigt und können über die Buttons unterhalb der Listen zur Testserie hinzugefügt (oder auch wieder entfernt) werden. Testsequenzen können bei Bedarf auch mehrfach in einer Testserie enthalten sein. Eine der Testserie bereits hinzugefügte Testsequenz wird im Listenfeld blau markiert. Sequenzen können, wenn nötig, auch mehrfach in einer Testserie vorkommen.

Die Reihenfolge im rechten Listenfeld legt die spätere Abarbeitungsreihenfolge der Sequenzen fest. Durch Klicken-und-Ziehen der Testsequenznamen kann die Reihenfolge angepasst werden.

Metadaten in den Projektinformationen müssen vor dem Speichern der Testserie vollständig ausgefüllt werden. Sie erscheinen später im Header des Testberichts.

Über den Button **„Edit Test Sequence“** („Testsequenzen bearbeiten“) können neue Testsequenzen angelegt oder bestehende bearbeitet werden. Ist eine Testsequenz momentan markiert, so wird sie beim Öffnen des Edit-Dialogs automatisch geladen. Ein Klick mit der mittleren Maustaste auf einen Testsequenzeintrag öffnet ebenfalls den Edit-Dialog für die entsprechende Testsequenz.

„Save“ bzw. **„Save As...“** („Speichern“, „Speichern unter...“) speichert die aktuelle Testserie, damit sie zur Testausführung zur Verfügung steht. Es bietet sich an, eindeutige Namen zu vergeben, damit die spätere Zuordnung von Testergebnissen zu Softwareversionen problemlos möglich ist. Dabei ist zu beachten, dass der Testbericht mit den Testergebnissen automatisch den Namen der zugrundeliegenden Testserie erhält.

Hinweis: Nur Testserien, die im konfigurierten Testserien-Ordner angelegt werden, sind später auch ausführbar. Siehe hierzu Abschnitte „Projektkonfiguration“ und „Standard-HIL-Projektstruktur“ im Anhang für weitergehende Informationen.

3.2.2 Testsequenzen bearbeiten

Der Testsequenz-Dialog, wie er in Abbildung 5 zu sehen ist, enthält im Normalfall bereits eine geladene Testsequenz, andernfalls kann über **„Load Sequence“** („Testsequenz laden“) problemlos eine andere bereits existierende Sequenz geladen werden.

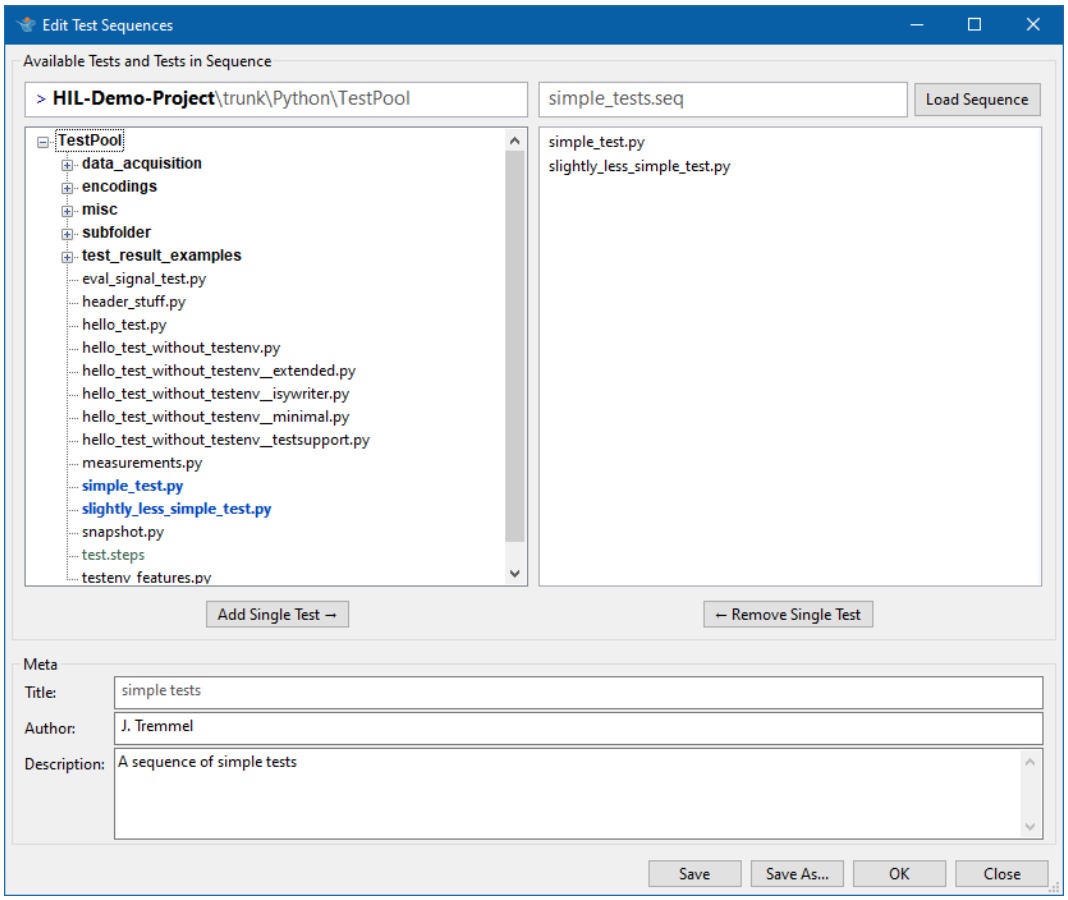


Abbildung 5: Testsequenz bearbeiten

Verfügbare Testskripte aus dem TestPool-Verzeichnis des aktuellen Projekts werden im linken Feld angezeigt und können über die Buttons unterhalb zur Testsequenz hinzugefügt oder auch wieder entfernt werden. Einzeltests können bei Bedarf auch mehrfach in einer Testsequenz enthalten sein.

Die Reihenfolge im rechten Listenfeld legt die spätere Abarbeitungsreihenfolge der Testskripte fest. Auch hier kann durch klicken-und-ziehen der Skriptnamen die Reihenfolge angepasst werden.

Metadaten zur Testsequenz – hier Autor und Beschreibungstext – müssen vor dem Speichern der Testserien vollständig ausgefüllt werden und erscheinen später im Header des Testsequenz-Abschnitts im Testbericht.

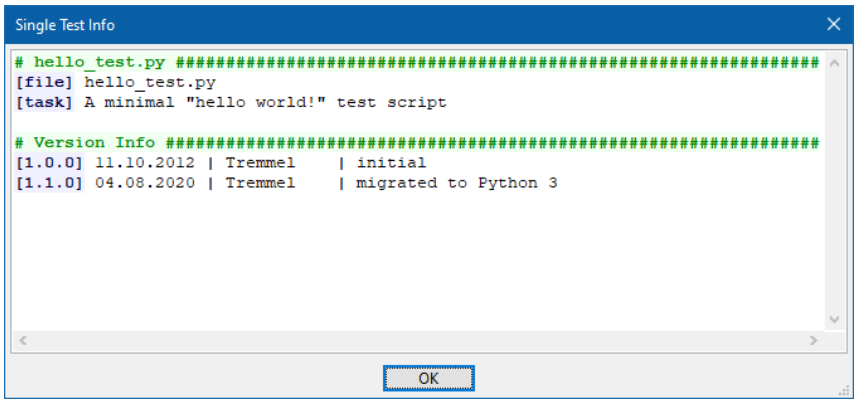


Abbildung 6: Informationen zum Testskript

Ein Klick mit der mittleren Maustaste auf einen Einzeltest öffnet einen Informationsdialog, wie er in Abbildung 6 zu sehen ist, der eine kurze Zusammenfassung der Informationen aus dem Dateiheader anzeigt.

„Save“ bzw. **„Save As...“** („Speichern“, „Speichern unter...“) speichert die aktuelle Testsequenz, damit sie in Testserien verwendet werden kann. Wird kein Titel für die Testsequenz festgelegt, so wird später in Testreports der Name der Testsequenz-Datei als Titel verwendet.

Hinweis: Nur Testsequenzen, die im konfigurierten Testsequenz-Order angelegt werden, sind später auch in der Teststruktur verfügbar. Siehe hierzu wieder die Abschnitte „Projektkonfiguration“ und „Standard-HIL-Projektstruktur“ im Anhang für weitergehende Informationen.

Hinweis: Die Anzahl der im TestPool angezeigten Verzeichnisebenen kann projektspezifisch angepasst werden. Siehe dazu die Option `test_pool_depth` im Abschnitt „Projektkonfiguration“.

3.3 Testdurchführung starten

Ist eine Testserie im Hauptfenster als **„Current Test Series“** („Aktuelle Testserie“) gewählt, so kann die Testdurchführung über **„Run All Tests“** oder **„Run Tests...“** gestartet werden („Alle Tests starten“ bzw. „Tests starten...“).

Hinweis: Damit Python-Testskripte ausgeführt werden können, sollte ein passender Python-Interpreter installiert sein und im Systempfad (PATH) eingetragen sein.

Für den Testablauf werden Einzeltests (Testskripte) über Testsequenzen zu einer Testserie zusammengefasst. Die Reihenfolge der Einzeltests innerhalb der Testsequenzen und die Anordnung der Sequenzen in der Testserie legen zusammen die Abarbeitungsreihenfolge im Testablauf fest.

- **„Run All Tests“** führt alle Testskripte der Testserie in der vorgegebenen Reihenfolge nacheinander aus.
- **„Run Tests...“** ermöglicht alternativ eine vorherige Auswahl, welche Tests gestartet werden sollen, wie dies in Abbildung 7 beispielhaft dargestellt ist. Dies ist insbesondere für Nachtests einzelner Testfälle hilfreich.

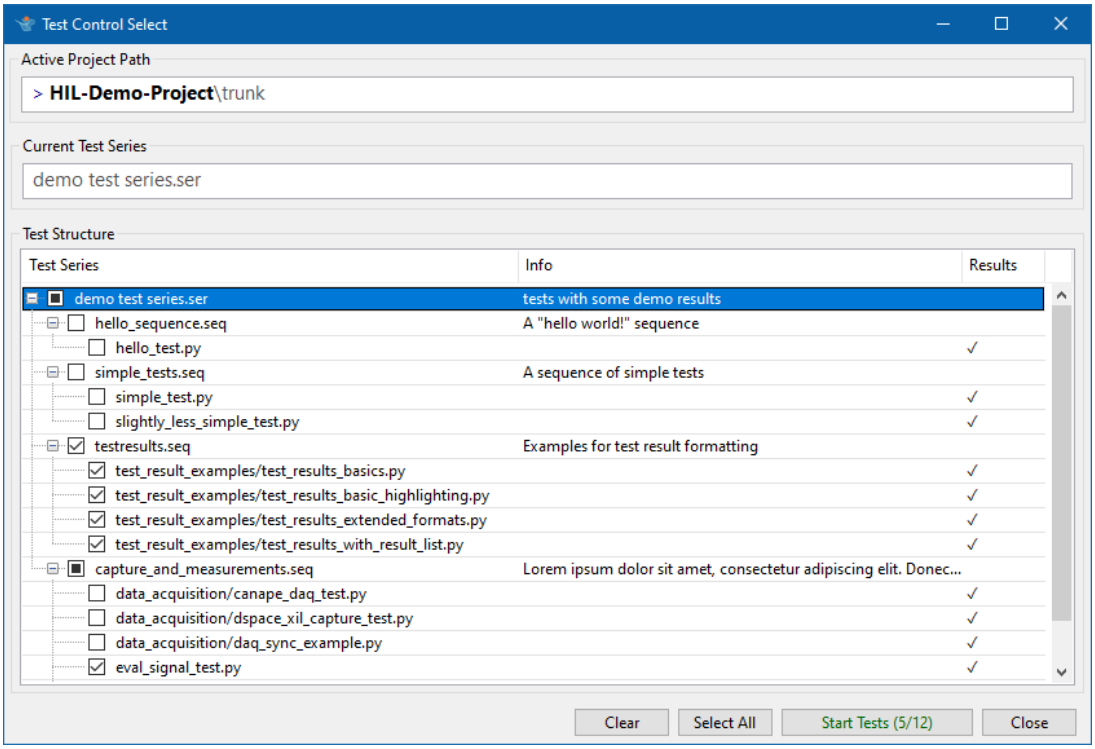


Abbildung 7: Vorauswahl von Einzeltests

Bereits bestehende Ergebnisse eines Einzeltests werden bei einem erneuten Ausführen des Tests durch die neuen Ergebnisse ersetzt. Die Ergebnisse anderer Tests bleiben davon unberührt.

Liegen für einen Test bereits Ergebnisse vor, so wird vor der Testausführung eine Sicherung der Einzelergebnisdatei angelegt (mit der Dateiendung .bak).

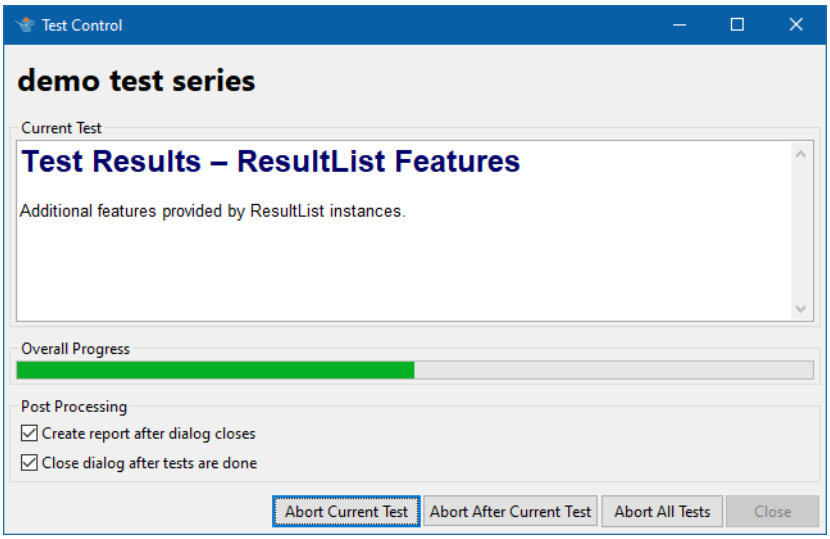


Abbildung 8: Laufende Testdurchführung

Der Testdurchführungs-Dialog in Abbildung 8 zeigt den Fortschritt des Testlaufs an und ermöglicht ein vorzeitiges Beenden des aktuellen Ablaufs:

- **„Abort Current Test“** („Aktuellen Test abbrechen“) bricht den aktuellen Test ab und setzt die Verarbeitung mit dem folgenden Test fort.
- **„Abort After Current Test“** („Nach diesem Test abbrechen“) wartet, bis der aktuelle Test fertig abgearbeitet ist und beendet dann den weiteren Testlauf.
- **„Abort All Tests“** („Alle Tests abbrechen“) unterbricht den Testablauf sofort.

Unter **„Post Processing“** („Nach dem Testlauf“) kann das Verhalten nach Ende des Testlaufs beeinflusst werden:

- Die Option **„Create report after dialog closes“** („Nach Schließen des Dialogs einen Testreport erstellen“) startet den aktuell ausgewählten Testreportgenerator automatisch mit den aktuellen Testergebnissen, nachdem der Dialog geschlossen wurde.
- Das Setzen von **„Close dialog after tests are done“** („Dialog schließen, wenn Tests beendet“) bewirkt ein automatisches Schließen des Testdurchführungs-Dialogs nach Testende.

Logausgaben der Testskripte erscheinen im Konsolenfenster und stellen – je nach Umsetzung der Testimplementierung – üblicherweise detailliertere Informationen zum momentanen Status des Tests dar. In Abbildung 9 sind entsprechende Ausgaben beispielhaft dargestellt. Die Ausgaben werden außerdem als Logdateien zusammen mit den Einzeltestergebnissen in einem Ordner unterhalb des konfigurierten Ergebnis-

ordners abgelegt (der Ordnername entspricht dem Testseriennamen). Weitere Informationen zur Ordnerstruktur sind im Anhang aufgeführt.

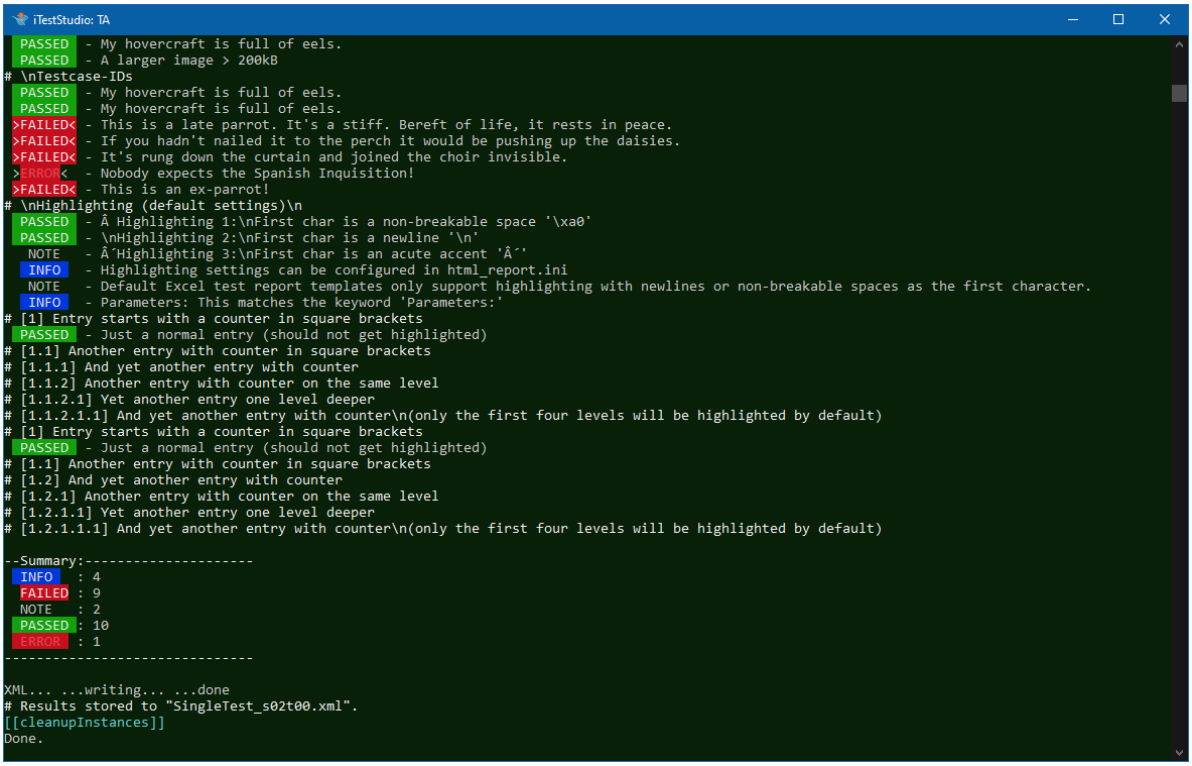


Abbildung 9: Test-Log / Konsole

Nachdem die Testserie abgearbeitet wurde, liegen Einzeltestergebnisse vor, die nun zu einem Testbericht/Testreport zusammengefasst werden können.

3.4 Testreports erstellen

3.4.1 Testreports für den aktuellen Testlauf

Nach dem Testlauf können die Ergebnisse zusammengestellt und im Anschluss daran ein Testbericht erstellt werden: Im Hauptfenster wird über **„Generate Report…“** („Testreport erstellen…“) die Testreporterstellung gestartet, wie sie in Abbildung 10 zu sehen ist.

Das Feld **„Series Info“** („Serien-Infos“) enthält eine Zusammenfassung der aktuellen Testserie. Neben den Basisinformationen wird für jeden Test der Serie angezeigt, ob bereits Ergebnisse vorliegen. Ein Bericht kann auch bei ausstehenden Testergebnissen erstellt werden, die betroffenen Testskripte erscheinen dann ohne Bewertung.

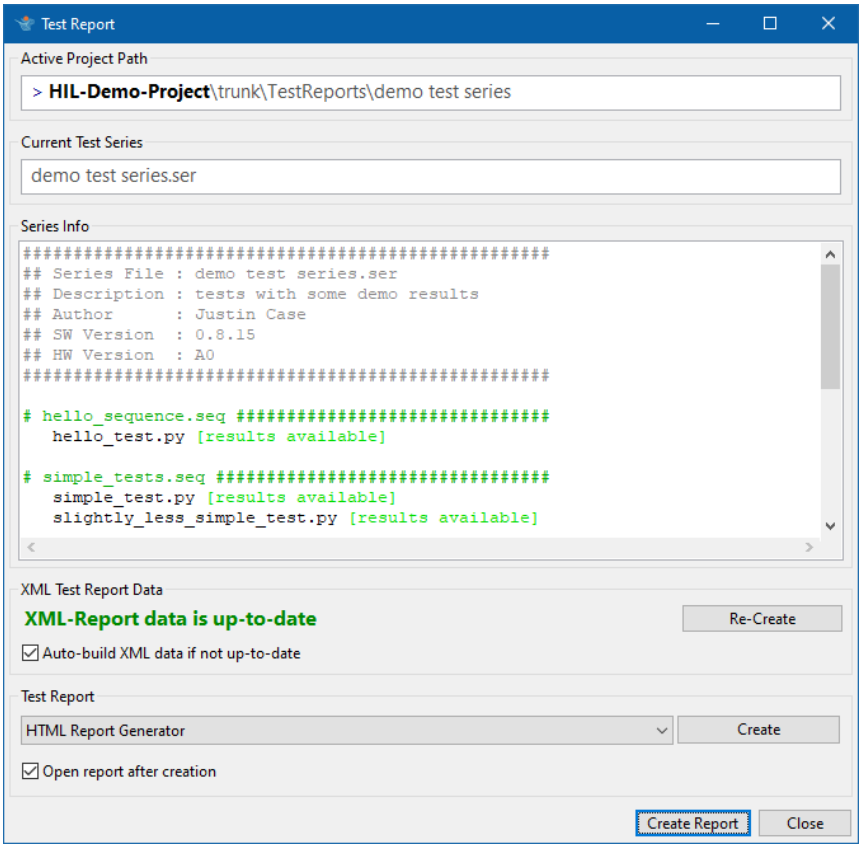


Abbildung 10: Testreports erstellen

Testberichte können in verschiedenen Formaten erstellt werden und bieten – je nach Format – Übersichten und Auswertungen an.

Es stehen primär folgende Reportgeneratoren zur Auswahl:

- **HTML-Reportgenerator:**
  Der gesamte Testbericht in einem einzigen HTML-Dokument mit eingebetteten Grafiken/Bildern.
- **HTML-Multidoc-Reportgenerator:**
  Der Testbericht wird aufgeteilt in mehrere untereinander verlinkte HTML-Dokumente gespeichert. Die Aufteilung in Unterdateien kann bei sehr umfangreichen Testergebnissen hilfreich sein, um Ladezeiten im Browser beim Betrachten der Ergebnisse zu verteilen.
- **PDF-Reportgenerator**
  Der Testbericht wird als ein mehrseitiges PDF-Dokument erstellt.
  Hinweis: Um diesen Reportgenerator verwenden zu können, wird zusätzlich eine lokale LaTeX-Umgebung im System-PATH benötigt.

Über **„Create Report“** („Report erstellen“) wird die Testberichterstellung mit dem ausgewählten Generator gestartet. Die Berichterstellung ist dabei zweistufig angelegt. Zuerst werden die vorliegenden Einzeltestergebnisse entsprechend der Teststruktur zu einem Gesamtergebnis zusammengefasst und als XML-Datei gespeichert.

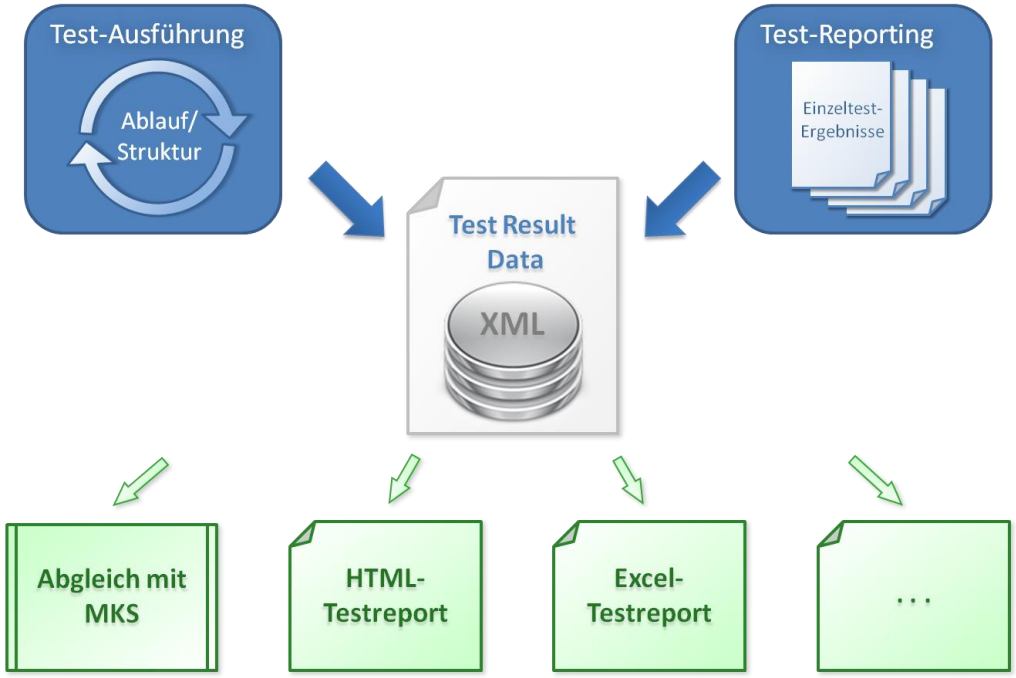


Abbildung 11: Testergebnisse, XML-Daten und Berichte

Im folgenden Schritt können dann aus den XML-Report-Daten ein oder auch mehrere Testberichte erstellt werden. Da die XML-Datei die kompletten Informationen zu Teststruktur und -ergebnissen enthält, können aus ihr auch zu einem späteren Zeitpunkt jederzeit weitere Testberichte neu erstellt werden. Zudem kann sie als Datenquelle für

externe Programme dienen, beispielsweise zur Synchronisierung der Testergebnisse mit Testmanagementwerkzeugen.

Anstelle von **„Create Report“** können XML-Daten und Testreport bei Bedarf auch über den jeweils zugeordneten „Create“-Button („Erstellen“) separat erstellt werden.

Ist die Option **„Auto-build XML-data if not up-to-date“** („XML-Daten neu erstellen, wenn veraltet“) gewählt, so wird sichergestellt, dass die XML-Daten bei Änderungen an Einzeltestergebnissen automatisch aktualisiert werden.

Die Option **„Open report after creation“** („Erstellten Testreport automatisch erstellen“) ermöglicht, den neu erstellten Testbericht über das mit seinem Dateityp verknüpfte Standardprogramm automatisch öffnen zu lassen.

3.4.2 Testreports zu bereits bestehenden XML-Reports

Außerhalb der aktuellen Testserie können Testreports auch einfach direkt aus bereits bestehenden XML-Reportdateien erstellt werden. Hierzu steht unter dem Menüpunkt **„Test Reporting“ → „Generate test reports...“** („Testreport“ → „Testreports erstellen...“) ein separater Testreportgeneratordialog zur Verfügung, siehe Abbildung 12.

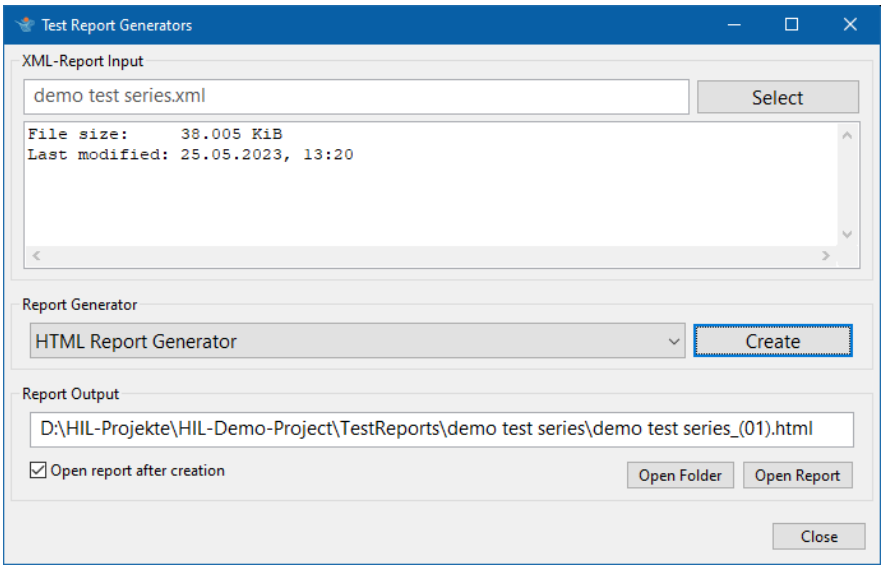


Abbildung 12: Testreports von bestehenden XML-Reports erstellen

Über **„Select“** („Auswählen“) kann eine XML-Reportdatei ausgewählt werden, für die dann mit **„Create“** („Erstellen“) der nebenstehend ausgewählte Reportgenerator gestartet wird. XML-Reportdateien können alternativ dazu auch per Drag&Drop aus dem Dateiexplorer in das XML-Input-Feld gezogen werden.

Im Textfeld darunter erscheint nach erfolgreichem Erstellen des Reports der Pfad zur erstellten Datei (bzw. – je nach Generator – der Pfad zum erstellten Ordner).

3.4.3 Vergleichen von Testreports

Um XML-Reportdateien miteinander zu vergleichen kann über dem Menüpunkt „**Test Reporting**“ → „**Generate diff reports...**“ („Testreport“ → „Diff-Report erstellen...“) ein kombinierter HTML-Bericht aus zwei oder mehreren Einzelreports erstellt werden (Abbildung 13). In einem solchen „Diff-Report“ werden die Inhalte der Einzelreports gegenübergestellt und Unterschiede bzw. Veränderungen der Ergebnisse hervorgehoben.

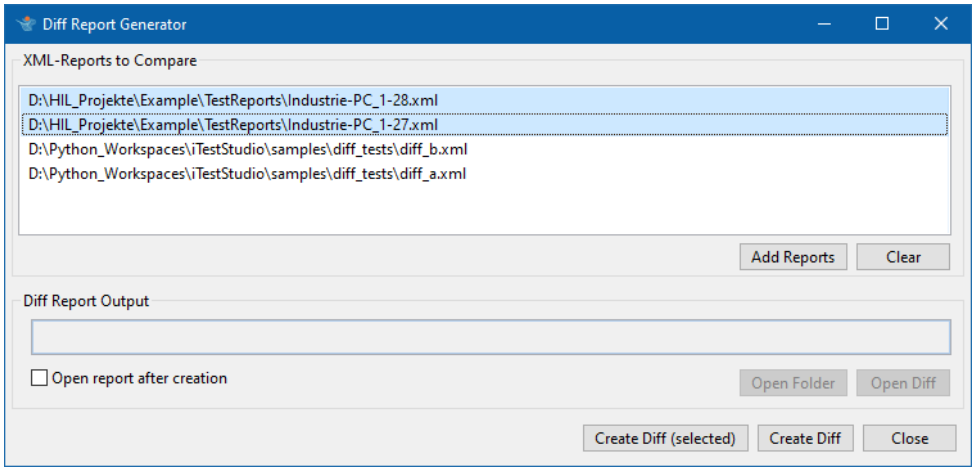


Abbildung 13: Diff-Reports zu mehreren bestehenden XML-Reports erstellen

Über **„Add Reports“** („Report hinzufügen“) können mehrere XML-Reportdateien zu der Vergleichsliste hinzugefügt werden, alternativ dazu auch per Drag&Drop aus dem Dateiexplorer. Die Reihenfolge in der Liste legt auch die Reihenfolge für den Vergleich fest, durch Klicken-und-Ziehen kann die Reihenfolge angepasst werden.

Mit **„Create Diff“** („Diff erstellen“) wird der Diff-Reportgenerator gestartet. Im Textfeld unterhalb der Vergleichsliste erscheint nach erfolgreichem Erstellen des Reports der Pfad zum erstellten Report.

Sind in der Vergleichsliste zwei oder mehr Reportdateien markiert, so kann mit **„Create Diff (selected)“** („Diff aus Auswahl erstellen") optional ein Diff-Report nur für die aktuelle Auswahl erstellt werden.

4 Anhang

4.1 Kommandozeilenoptionen

Kommandozeilenoptionen bzw. -parameter sind – wie der Name schon nahelegt – optional. Sie können z. B. verwendet werden, um das iTestStudio TA in einem automatisierten Modus zu starten, in dem keine Benutzerinteraktion nötig ist.

Option/Parameter	Beschreibung
`--project PROJECT`	Legt das aktive HIL-Projektverzeichnis fest. **Beispiel:** iTestStudio TA mit einem Projekt in D:\HIL-Projekte\Demo-Projekt als aktives HIL-Projekt starten: `iTestStudioTA --project D:\HIL-Projekte\Demo-Projekt` Ohne Angabe der Option wird das iTS mit dem zuletzt gewählten Projektpfad gestartet.
`--series SERIES`	Legt die aktive Testserie fest. **Beispiel:** iTestStudio TA mit einer vorausgewählten Testserie „DemoSeries.ser“ starten. Die Testserie muss hierzu im aktuellen Projekt im konfigurierten Testserien-/Teststrukturordner vorhanden sein. In der Testserie referenzierte Testsequenzen werden bei Bedarf nachgeladen. `iTestStudioTA --series DemoSerie.ser` Ohne Angabe der Option wird die im Projekt zuletzt verwendete Testserie bzw. Teststruktur gewählt.
`--struct STRUCT`	Legt die aktive Teststruktur fest. Teststrukturen definieren Testabläufe und beinhalten die Informationen einer Testserie und aller darin referenzieren Testsequenzen in einer Datei. **Beispiel:** iTestStudio TA mit einer vorausgewählten Teststruktur „DemoStruct.itstruct“ starten. Die Teststruktur muss dazu im aktuellen Projekt im konfigurierten Testserien-/Teststrukturordner vorhanden sein. `iTestStudioTA --struct DemoStruct.itstruct` Ohne Angabe der Option wird die im Projekt zuletzt verwendete Testserie bzw. Teststruktur gewählt. **Hinweis:** Wird eine Testserie (*.ser) als Teststruktur geladen, so werden im automatisierten Modus („--runall“) keine Tests ausgeführt, da die übergebene Struktur direkt verwendet wird und die in einer Testserie nur referenzierten Testsequenzen nicht nachgeladen werden.

`--runall`	Startet das iTestStudio TA im automatisierten Modus und führt alle Tests in der aktuellen Testserie bzw. Teststruktur aus. Nach Ende des Testlaufs wird das Programm beendet.
`--runselection`	Startet das iTestStudio TA im automatisierten Modus und führt die ausgewählten Tests der aktuellen Testserie bzw. Teststruktur aus. Nach Ende des Testlaufs wird das Programm beendet.
`--ini OVERRIDE_INI`	Startet das iTestStudio TA mit einer benutzerspezifizierten Konfigurationsdatei. **Beispiel:** iTestStudio mit einer angepassten Konfiguration „its_custom.ini“ starten. `iTestStudioTA --ini D:\settings\its_custom.ini` Die Einstellungen in der übergebenen Konfiguration überlagern die Einstellungen aus den Programmeinstellungen.
`--help`	Zeigt eine Liste aller verfügbaren Optionen/Parameter an.
`--version`	Gibt die iTestStudio-TA-Version aus.

4.1.1 Rückgabewerte

Im automatisierten Modus (also bei Aufrufen mit `--runall` oder `--runselection`) gibt der iTestStudio-TA-Prozess einen Rückgabewert zurück („Exit Status“).

Folgende Statuswerte sind definiert:

Exit Status/Code	Beschreibung
`0`	Programm normal beendet
`1`	Projektordner nicht vorhanden
`2`	Übergebene Testserien-/Teststrukturdatei nicht verfügbar
`3`	Ungültiger Projektordner (nicht lesbar oder Struktur entspricht nicht der Konfiguration)
`4`	Keine Testserie/Teststruktur ausgewählt
`10`	Lizenzüberprüfung fehlgeschlagen

4.1.2 Beispiel

Es soll ein Testlauf im Projekt „D:\HIL_Projekte\HIL-Demo-Project\trunk“ automatisch gestartet und aus den Ergebnissen mehrere verschiedene Testreports erstellt werden. Im Ordner „TestStructure“ des HIL-Projekts liegt dazu bereits eine Teststrukturdatei „demo test series.itstruct“, in der der gesamte Testablauf festlegt und eine Auswahl an zu testenden Einzeltests definiert ist.

Eine benutzerspezifische Konfigurationsdatei „custom_iteststudio.ini“ mit folgendem Inhalt wurde erstellt:

```
# Path to HIL project
project_path: D:\HIL_Projekte\HIL-Demo-Project\trunk

[TestReport]
# Test report generator(s) to trigger after test run is complete
# Available: HTML, Multidoc, PDF, Allure, Text, XUnit, DXL
generator: HTML, Allure, PDF
```

Wird im Abschnitt „TestReport“ bei „generator“ eine kommaseparierte Liste mit mehreren Generatornamen angegeben, so werden später nacheinander alle aufgeführten Generatoren aufgerufen. Ohne diese Angabe wird einfach der zuletzt im Projekt benutzte Generator verwendet.

Hinweis: Da die Konfigurationsdatei im gegebenen Beispiel bereits den Pfad zum HIL-Projekt enthält, muss dieser nicht zusätzlich über die Kommandozeile festgelegt werden. Ansonsten könnte dies durch Hinzufügen des folgenden Parameters geschehen:

```
--project "D:\HIL_Projekte\HIL-Demo-Project\trunk"
```

Durch den Kommandozeilenaufruf

```
> iTestStudioTA.exe --ini "D:\HIL_Projekte\custom_iteststudio.ini"
  --struct "demo test series.itstruct" --runselection
```

wird schließlich der Testlauf mit den ausgewählten Tests gestartet.

Nach Abschluss des Testlaufs wird aus den Einzeltestergebnissen ein XML-Report erstellt und danach alle konfigurierten Reportgeneratoren angestoßen. Nach dem Erstellen der Testreports beendet sich das iTestStudio TA (mit Exit Status 0).

4.2 Programmkonfiguration

Im „settings“-Ordner der iTestStudio-TA-Installation ist die Programmkonfigurationsdatei **„iteststudio.ini“** mit den Grundeinstellungen zu finden. Benutzerspezifische Einstellungen wie zuletzt gewählte Dateien oder Fensterpositionen werden automatisch in einer weiteren **„iteststudio.ini“** im Benutzerverzeichnis gespeichert.

Beispielinhalt der Konfigurationsdatei:

```
# Initial setting for "last selected project" path
project_path: D:\HIL_Projekte

# User interface localization.
# Available locales: DE, EN
locale: EN

[INTERPRETER]
# Manual settings for (Python) interpreter.
# If not set manually, the system's default Python interpreter will be used.
# Only use this setting if absolutely needed, default settings should be fine.
# executable = C:\Tools\Python27\python.exe

[ENVIRONMENT]
# Manual settings for (extra) environment variables that will be handed on
# to the TestRunner process.
# As before, only use this feature if absolutely needed.
# PYTHONHOME = C:\Tools\Python27
# PYTHONPATH = C:\Tools\Python27;C:\Tools\Python27\lib;C:\Tools\Python27\DLLs;
# PATH       = C:\Tools\Python27;C:\Windows\System32
```

Hinweis:

Für Sonderfälle kann über die Abschnitte INTERPRETER und ENVIRONMENT ein Python-Interpreter konfiguriert werden, der vom systemweit installierten Standard-Python abweicht. Dieser wird dann für die Testausführung aller Projekte verwendet.

Analog dazu kann dies aber auch in der jeweiligen Projektkonfigurationsdatei (**„hil_project.ini“** im Projektordner) erfolgen, dann kommt die Sondereinstellung nur für das jeweilige Projekt zur Anwendung.

4.3 Projektkonfiguration

Im HIL-Projektordner ist auf der obersten Ebene die Projektkonfigurationsdatei **„hil_project.ini“** zu finden, in der die Projektordnerstruktur angepasst werden kann.

Beispielinhalt der Konfigurationsdatei:

```
[FOLDERS]
# python script folders
test_pool  = Python/TestPool
test_libs  = Python/TestLibraries
```

```
# test results/reports
test_reports = TestReports
test_results = TestReports/SingleTestResults

# test structure data
test_series    = TestStructure
test_sequences = TestStructure/TestSequences

# number of sub-folder levels below test pool to show in EditSequences
# range 0..12
test_pool_depth = 2

[RESULTS]
test_result_save_name        = SingleTest.xml
manual_test_result_save_name = SingleTestManual.xml

[TestToolkit]
locale = de

[ENVIRONMENT]
# PYTHONHOME = C:\Tools\Python27
# PYTHONPATH = C:\Tools\Python27;C:\Tools\Python27\lib;C:\Tools\Python27\DLLs;
# PATH = C:\Tools\Python27;C:\Windows\System32;

[INTERPRETER]
# executable = C:\Tools\Python27\python.exe
```

Hinweise:

- Alle Pfadangaben sind relativ zum Projektordner anzugeben.
- Die „locale“-Einstellung im Abschnitt TestToolkit legt fest, in welcher Sprache die Beschreibungstexte in den Testfunktionen erstellt werden. Derzeit sind Lokalisierungen für Englisch (EN) und Deutsch (DE) verfügbar.

Für die Abschnitte INTERPRETER und ENVIRONMENT siehe den Hinweis im vorhergehenden Kapitel zur Programmkonfiguration.

In der Projektkonfigurationsdatei können auch projektspezifische Standardwerte für die Metadaten neuer Testsequenzen/Testserien hinterlegt werden:

```
[Default Series Meta Data]
author         = Blaise Holder
department     = Public Relations
project        = Plankton Extractor
customer       = Soylent Corporation
sw_version     = 4.7.11
hw_version     = A4
description    = Lorem ipsum dolor sit amet, consectetur adipiscing elit.
                 Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas
                 congue ligula ac quam viverra nec consectetur ante hendrerit.

[Default Sequence Meta Data]
author         = Gene Eric Person
description    = Lorem ipsum dolor sit amet, consectetur adipiscing elit.
                 Donec a diam lectus. Sed sit amet ipsum mauris.
```

4.3.1 Beispiel: venv für ein HIL-Projekt konfigurieren

Soll ein HIL-Testprojekt einen in ein virtual environment abgekapselten Python-Interpreter verwenden, so kann dies in der jeweiligen **hil_project.ini** (im Projektordner) konfiguriert werden.

Mit einem angelegten Python virtual environment, z.B. über

```
> python -m venv C:\Tools\py312_venv --system-site-packages
```

kann im Konfigurationsabschnitt INTERPRETER direkt die **python.exe** aus dem venv als für die Tests zu verwendende Interpreter festgelegt werden:

```
[INTERPRETER]
executable = C:\Tools\py312_venv\Scripts\python.exe
```

Die python.exe im venv wird dann für die Testausführung im iTestStudio-TA für dieses Projekt verwendet.

4.3.2 Beispiel: Einen abweichenden Pythoninterpreter konfigurieren

Soll bei mehreren installierten Pythonversionen ein anderer als der systemweit eingerichtete Standard-Python-Interpreter für ein HIL-Projekt verwendet werden (und ein venv ist nicht gewünscht), so kann dies in der **hil_project.ini** im jeweiligen Projektordner manuell konfiguriert werden.

Neben dem Interpreter an sich (wie beim venv-Ansatz) sind aber noch zusätzliche Umgebungsvariablen anzupassen, damit die gewünschte Pythonversion eigenständig lauffähig ist.

Als Beispiel mit einer bestehenden Python-Installation in **C:\Tools\Python27**:

```
[ENVIRONMENT]
PYTHONHOME = C:\Tools\Python27
PYTHONPATH = C:\Tools\Python27;C:\Tools\Python27\lib;C:\Tools\Python27\DLLs;
PATH = C:\Tools\Python27;C:\Windows\System32;

[INTERPRETER]
executable = C:\Tools\Python27\python.exe
```

Hinweis: Hierbei wir der normale System-PATH für die Ausführung der Testskripte ersetzt. Aus Testskripten heraus aufgerufene externe Programme erben diesen PATH, daher kann es gegebenenfalls nötig werden, die Einträge in PATH noch um weitere Pfade zu ergänzen.

4.3.3 Beispiel: TestHandler für ein Projekt definieren

TestHandler sind konfigurierbare Testskripte, die im Testlauf eine Testdefinitionsdatei übergeben bekommen und ein Testergebnis zu dieser Testdefinition erstellen und ablegen.

Es können in einem Projekt TestHandler für spezifische Dateiendungen angelegt werden. Dateien im TestPool, die diese Endungen haben, können dann wie normale Python-Testskripte einer Testsequenz hinzugefügt werden. Anstelle der Datei selbst wird jedoch das zugeordnete TestHandler-Skript ausgeführt, dem die Datei als Testdefinition in den Aufrufparametern übergeben wird.

In der hil_project.ini im Testprojektordner:

```
[TestHandlers]
# Test handler/runner scripts that process custom test definition data
# ("handler" to make them more distinct from the main test_runner.py)
#
# Examples:
#   * a test handler for TTCN-3 test definitions
#   * a test handler for custom keyword-based tests
#
# Entry format:
# file-extension: path_to_handler_python_script (below test project)
#ttcn:   Python/TestPool/_test_handler_ttcn3.py
#kwtest: Python/TestPool/_test_handler_kwtest.py
steps:  Python/TestPool/_test_handler_demo.py
```

Damit werden im Beispiel Dateien mit der Endung „.step“ im Edit-Sequences-Dialog als „Tests“ anwählbar. Im Testlauf wird dann das **_test_handler_demo.py**-Skript ausgeführt und erhält den Pfad zur .step-Datei als Übergabeparameter.

Anhang 4.6 Test-Handler-Skripte im iTestStudio beschreibt den grundsätzlichen Aufbau von Test-Handlern

4.4 Standard-HIL-Projektstruktur

Die im vorhergehenden Beispiel konfigurierte Standardstruktur stellt sich im Dateisystem wie folgt dar (inklusive Standardordner einiger typischer externer Werkzeuge):

- **Projektordner/**
  - hil_project.ini
  - **CANape/**
    - canape.ini
    - ecu.a2l
    - layout.cna
  - **Documentation/**
    - hil_schematics.pdf
    - TestToolkit.chm
  - **Model/**
    - hil_model.slx
    - hil_model.trc
    - hil_model.sdf
  - **ControlDesk/**
    - hil_model.cdx
  - **Python/**
    - **TestLibraries/**
      - **ttk_base/**
      - **ttk_bus/**
      - **ttk_checks/**
      - **ttk_daq/**
      - **ttk_tools/**
      - hil_variables.py
      - cal_variables.py
    - **TestPool/**
      - _automation_wrapper_.py
      - testscript.py
  - **TestReports/**
    - **SingleTestResults/**
  - **TestStructure/**
    - test_series.ser
    - **TestSequences/**
      - test_sequence.seq

4.5 Testskripte im iTestStudio

Tests im iTestStudio sind Pythonskripte, die mit dem konfigurierten Pythoninterpreter ausgeführt werden. Die Testskripte werden im Testlauf nacheinander in jeweils eigenen Prozessen ausgeführt und müssen am Ende das Testergebnis in einer Datei im iTS-Ergebnisformat speichern.

4.5.1 Schnittstelle

Übergabeparameter: Ablagepfad zum Testergebnis

Arbeitsverzeichnis: TestPool-Ordner

4.5.2 Ergebnisformat

Ergebnisformat: iTS-XML. Ein Singletest-XML-Result enthält einen Abschnitt mit Metadaten zum ausgeführten Test sowie eine Liste von Testschrittergebnissen, jeweils mit Beschreibung und Bewertung (und optional weiteren Einträgen).

```
<?xml version="1.0" encoding="UTF-8"?>
<singletest source="hello_test.py">
    <meta>
        <title>hello test</title>
        <description>A minimal "hello world!" test script</description>
        <author>Tremmel</author>
        <date>04.08.2020</date>
        <version>1.1.0</version>
        <last_revision>migrated to Python 3</last_revision>
        <starttime>2024-04-19T18:29:14</starttime>
        <endtime>2024-04-19T18:29:16</endtime>
    </meta>
    <results>
        <result>
            <description>Hello World!</description>
            <verdict>PASSED</verdict>
            <deltatime>1.511</deltatime>
        </result>
    </results>
</singletest>
```

Abbildung 14: Beispiel: XML-Testergebnis

In Testskripten muss die XML-Ergebnisstruktur normalerweise nicht manuell angelegt werden, sondern wird über Methoden der TestSupport-Bibliotheken automatisch aus den Einträgen der Testresults-Liste im Test erzeugt; die Metadaten werden dabei in erster Linie aus dem Testskript-Header befüllt.

4.5.3 Beispielskript mit TestEnv

Ein minimales Testskript mit einem Testschritt, der mit PASSED bewertet ist. Das Speichern der Testergebnisse erfolgt in der breakdown-Methode des Test-Environments, das aus dem „_automation_wrapper_“-Basisskript importiert wurde.

```
#*******************************************************************************
# -*- coding: utf-8 -*-
# File: hello_test.py
# Task: A minimal "hello world!" test script
#
# Copyright 2012 - 2020 iSyst Intelligente Systeme GmbH
#*******************************************************************************
#****************************** Version ****************************************
#*******************************************************************************
# Ver.    | Date       | Name     | Description
#-------------------------------------------------------------------------------
# 1.0.0   | 11.10.2012 | Tremmel  | initial
# 1.1.0   | 04.08.2020 | Tremmel  | migrated to Python 3
#*******************************************************************************
from _automation_wrapper_ import TestEnv
testenv = TestEnv()

try:
    testenv.setup()

    testresult = testenv.getResults()

    testresult.append(["Hello World!", "PASSED"])

finally:
    testenv.breakdown()

print("Done.")
```

Abbildung 15: Ein Hello-World-Beispieltestskript

Ein projektspezifisch angepasstes **TestEnv** wird im Modul _automation_wrapper_.py bereitgestellt, das sich im TestPool-Ordner befindet.

Neben dem Bereitstellen des TestEnvs initialisiert der zentrale Wrapper auch die die Projekt- und Bibliotheksimportpfade für das aktuelle Projekt, sodass Module aus **TestLibraries** (im aktuellen Projektordner) und **TestSupport** (aus der iTS-Installation) in den Testskripten zur Verfügung stehen.

```
#*******************************************************************************
# -*- coding: utf-8 -*-
# File: _automation_wrapper_.py
# Task: Basic functionality and definitions for test automation
#
# Copyright 2020 iSyst Intelligente Systeme GmbH
#*******************************************************************************
#******************************** Version **************************************
#*******************************************************************************
# Ver.    | Date       | Name      | Description
#-------------------------------------------------------------------------------
# 1.0.0   | 05.04.2020 | Tremmel   | initial, basic environment
#*******************************************************************************
import os
import sys
import inspect

# initialize path to test support libraries ###############################
osp = os.path
_test_support_path = osp.normpath(os.path.join(
    os.getenv("ITESTSTUDIO_PATH", ""), "libraries",
    "test_support%d%d.pyz"%sys.version_info[0:2] # select matching libs
))
# derive project base path from this script's folder ######################
_test_pool_path = osp.abspath(inspect.currentframe().f_code.co_filename)
_test_pool_path = osp.dirname(_test_pool_path)
_base_project_path = osp.dirname(_test_pool_path)
_base_project_path = osp.dirname(_base_project_path)

# Test Support / Environment ##############################################
if _test_support_path not in sys.path:
    sys.path.insert(1, _test_support_path)
from test_environment import TestEnvironmentBase, getConfig

# get configured folder paths from hil_project.ini
config = getConfig(_base_project_path)
assert _test_pool_path == config.getAbsPath("FOLDERS", "test_pool")
_test_libs_path = config.getAbsPath("FOLDERS", "test_libs")

# add project Libraries to module search path
if _test_libs_path not in sys.path:
    sys.path.append(_test_libs_path)
```

Abbildung 16: Beispiel: _automation_wrapper_ – Initialisierung der Pfade

Auf der Initialisierung der Projekt- und Bibliothekspfade folgt im Automation-Wrapper die **TestEnv**-Klasse, die das Basis-Testenvironment um Funktionalitäten für das jeweilige Projekt erweitert:

In **setup()** wird die Testumgebung initialisiert, was üblicherweise das Herstellen einer Verbindung zum Echtzeitsystem und ein Anlegen eines HIL-Variablen-Containers (meist in TestLibraries/hil_variables.py) beinhaltet. Weiteren Schnittstellen/Tool-Anbindungen werden normalerweise erst bei Bedarf im Lauf eines Tests angelegt.

Als Gegenstück dazu wird in der Methode **breakdown()** die Umgebung wieder weggeräumt und alle verwendeten Schnittstellen wieder geschlossen. Das breakdown() der Basisklasse übernimmt das Speichern der Testergebnisse.

Weitere Methoden können nach Bedarf ergänzt werden, so z.B.

- Getter für weitere Schnittstellen/Variablencontainer (getHil, getCal, getRecorder)
- Hilfsfunktionen zum ein-/ausschalten des Prüflings (startupDUT, shutdownDUT)

```
# ############################################################################
class TestEnv(TestEnvironmentBase):
    """ Test environment automation access helper/wrapper.
        Extend/adapt as needed for project needs.
    """
    def __init__(self):
        super().__init__(_base_project_path)

    # ########################################################################
    def setup(self):
        """ Test Setup: initialize test environment. """
        super().setup()
        # setup real-time environment and any other connections here,
        # if they will be needed/used in all test scripts

    # ########################################################################
    def breakdown(self):
        """ Test breakdown: cleanup & housekeeping. """
        # the base's breakdown() will handle, among other things,
        # storing of current test results
        super().breakdown()
        # close/cleanup connections / references here

    # add getters for tools/interfaces as needed, e.g. getHil/getRecorder

    # add utility methods as needed, e.g. startupDUT/shutdownDUT
```

Abbildung 17: Beispiel: _automation_wrapper_ – angepasstes TestEnv

4.5.4 Beispielskript ohne TestEnv

Soll kein TestEnvironment verwendet werden, so muss ein Testskript selbst das Speichern der Ergebnisse im iTS-XML-Ergebnisformat übernehmen. Ein auf das Wesentliche reduziertes „Hello World“-Testskript (mit einem Testschrittergebnis) könnte dann wie folgt aussehen:

```
#*******************************************************************************
# -*- coding: utf-8 -*-
# File: hello_test_without_testenv.py
# Task: A "hello world!" test script without using a TestEnv, TestSupport
#       or TestToolkit libraries.
#
# Copyright 2024 iSyst Intelligente Systeme GmbH
#*******************************************************************************
#****************************** Version ****************************************
#*******************************************************************************
# Ver.    | Date       | Name     | Description
#-------------------------------------------------------------------------------
# 1.0.0   | 22.03.2024 | Tremmel  | initial
#*******************************************************************************
```

```
"""
Test scripts running in iTestStudio-TA will get the file path to the result
xml file as first positional parameter (i.e. sys.argv[1]).
They are expected to write any test results to this file path in iTS Test
Result XML format
"""
import sys
result_xml = """<?xml version="1.0" encoding="UTF-8"?>
<singletest source="hello_test_without_testenv.py">
    <meta>
        <title>hello test (minimal, without testenv)</title>
        <description>A minimal "hello world!" test script</description>
        <author>Tremmel</author>
        <date>22.03.2024</date>
        <version>1.0.0</version>
        <last_revision>initial</last_revision>
    </meta>
    <results>
        <result>
            <description>Hello World!</description>
            <verdict>PASSED</verdict>
            <testcase_id>hello</testcase_id>
        </result>
    </results>
</singletest>
"""
# Get path to test result file. This may be empty if script is executed
# outside of iTS.

# Note: When using a TestEnv (i.e. derived from TestEnvironmentBase),
#       a fallback single test result path (below the SingleTestResults
#       folder) will be used if no path has been supplied.
output_path = sys.argv[1] if len(sys.argv) > 1 else None
if output_path:
    with open(output_path, "w", encoding="utf8") as out:
        out.write(result_xml)
else:
    print("# result file path was not supplied, result data is:")
    for num, line in enumerate(result_xml.splitlines(), 1):
        print(f"{num:3d} | {line}")

print("Done.")
```

Abbildung 18: Minimales Beispieltestskript ohne TestEnv

4.6 Test-Handler-Skripte im iTestStudio

Test Handler im iTestStudio sind Pythonskripte, die mit dem konfigurierten Pythoninterpreter ausgeführt werden.

Sind Handler-Skripte für ein Testprojekt konfiguriert (vgl. 4.3.3 „Beispiel: TestHandler für ein Projekt definieren“), so können die für sie definierten Dateitypen als Tests in Testsequenzen angegeben werden. Im Testlauf wird dann der für diese Dateien zuständige Test Handler aufgerufen, der die Testdefinitionsdatei interpretiert und die Ergebnisse daraus im iTS-Einzeltestergebnisformat abspeichert.

4.6.1 Schnittstelle

Übergabeparameter:

- Ablagepfad zum Testergebnis
- Pfad zur Testdefinitionsdatei

Arbeitsverzeichnis: TestPool-Ordner

4.6.2 Beispielskript

```
#*******************************************************************************
# -*- coding: utf-8 -*-
# File: _test_handler_demo.py
# Task:  demonstration of a "test handler" script, a runner/interpreter for
#        custom test scripts. As basic example, this test handler will just
#        create one result for each non-empty line in the test definition
#        file and use a default testresult list instead of generating an
#        XML result directly.
#
# Copyright 2024 iSyst Intelligente Systeme GmbH
#*******************************************************************************
#****************************** Version ****************************************
#*******************************************************************************
# Ver.    | Date       | Name     | Description
#-------------------------------------------------------------------------------
# 1.0.0   | 19.04.2024 | Tremmel  | initial
#*******************************************************************************
import os
import sys
from _automation_wrapper_ import TestEnv
from isytest import isyresults # @UnresolvedImport (TestSupport)
testenv = TestEnv()

# Test Handler Arguments:
# argv[0] test handler path
# argv[1] path to results file (where test results have to be stored)
# argv[2] test definition file path

try:
    result_output_path   = testenv.getResultsFilePath()
    # or without testenv (no fallback result path, though):
    # result_output_path = sys.argv[1] if len(sys.argv) >= 2 else None
    test_definition_path = sys.argv[2] if len(sys.argv) >= 3 else ""
```

```
    testresult = testenv.getResults()
    try:
        with open(test_definition_path, "r", errors="replace") as testdef:
            for line in testdef:
                line = line.strip()
                if line:
                    testresult.append([line, "INFO"])
    except OSError as ex:
        testresult.append([f"{type(ex).__name__}: {ex}", "ERROR"])

finally:
    # store test results, use standard testresults to results data mapping
    title = os.path.splitext(os.path.basename(test_definition_path))[0]
    isyresults.storeTestResultsFile(
        testresult      = testresult,
        result_path     = result_output_path,
        testscript_path = test_definition_path,  # for result "source"
        testpool_path   = testenv.testpool_path, # result "source" base

        # meta data should be extracted from custom test definition file
        additional_meta = {
            'TITLE':          title.replace("_", " "),
            'DESCRIPTION':    "Lorem Ipsum dolor sit amet",
            'AUTHOR':         "N.N.",
            'DATE':           "00.00.0000",
            'VERSION':        "0.0",
            'LAST_REVISION': "",
        },
        # don't try to extract meta data from testscript_path
        get_header_meta = False,
    )
```

Abbildung 19: Test-Handler-Beispiel

4.7 Ablagestruktur für Testergebnisse

4.7.1 Klassische Struktur

Einzelergebnisse und Logdateien werden in einem nach der Testserie benannten Unterordner im Ordner für Einzeltestergebnisse abgelegt. Testberichte werden direkt im „TestReports“-Ordner erstellt.

```
TestReports/
    SingleTestResults/
        # results from test runs outside of a defined test series
        SingleTestManual - hello_test.xml
        # results from test series run + logs, plots, images, ...
        <test_series_name>/
            some_image.png
            SingleTest_s00t00_2020-02-28T17-02-19.log
            SingleTest_s00t00.xml
    # complete xml report, aggregated from single results
    <test_series_name>.xml
```

4.7.2 Container-Struktur

Ab iTestStudio TA 5.0.0: Alle Daten eines Testlaufs werden in einem gemeinsamen, nach der Testserie benannten Unterordner des „TestReports“-Ordners abgelegt.

```
TestReports/
    SingleTestResults/
        # results from test runs outside of a defined test series
        SingleTestManual - hello_test.xml
    <test_series_name>/
        # container for result data of a specific test series
        data/
            # alternative folder for data objects (plots, images, ...)
            some_image.png
        logs/
            # log files created during test run
            SingleTest_s00t00_2020-02-28T17-02-19.log
        results/
            # single test results created by test scripts
            SingleTest_s00t00.xml
        # active test structure, will be stored on start of test run
        <test_series_name>.itstruct
        # complete xml report, aggregated from single results
        <test_series_name>.xml
```

4.7.3 Konfiguration

Die zu verwendende Ablagestruktur kann auf Projektebene in der hil_project.ini gewählt werden:

```
[RESULTS]
# container or classic
storage_structure = container
```

4.8 Aufbau der Lizenzdatei

Der Inhalt der iTestStudio-Lizenzdatei („**license.its**“) hat folgende Struktur:

```
[Dongle-ID] [Feature] [Version] [Ablaufdatum] [Prüfsumme]
```

Wobei gilt:

1. Eine Zeile pro lizensiertes Feature
2. Leerzeilen sowie Zeilen beginnend mit „#“ werden ignoriert.

Beispiel:

```
# Lizenzen:
D-USB-2014-B-1234 iTestStudio 4.1.x 2015-09-01 6C5F6DF4DF78626133D5BBE56E6B301D2798206F448A02C1FC24D16D5ACC70D9
D-USB-2014-B-1234 TestToolkit 1.4.x 2015-09-01 9FD920BB422A36C6D8C81D213836E016AC6FB036CC6F6989BF4E457EDD4CA9C6
```

Hinweis: Zusätzliche oder aktualisierte Lizenzeinträge können einfach durch anfügen der Zeilen hinzugefügt werden.

4.9 Abbildungsverzeichnis

Abbildung 1: Komponenten im iTestStudio 4
Abbildung 2: Hauptfenster 7
Abbildung 3: Teststruktur 9
Abbildung 4: Testserie bearbeiten 10
Abbildung 5: Testsequenz bearbeiten 12
Abbildung 6: Informationen zum Testskript 13
Abbildung 7: Vorauswahl von Einzeltests 14
Abbildung 8: Laufende Testdurchführung 15
Abbildung 9: Test-Log / Konsole 16
Abbildung 10: Testreports erstellen 17
Abbildung 11: Testergebnisse, XML-Daten und Berichte 18
Abbildung 12: Testreports von bestehenden XML-Reports erstellen 19
Abbildung 13: Diff-Reports zu mehreren bestehenden XML-Reports erstellen 20
Abbildung 14: Beispiel: XML-Testergebnis 29
Abbildung 15: Ein Hello-World-Beispieltestskript 30
Abbildung 16: Beispiel: _automation_wrapper_ – Initialisierung der Pfade 31
Abbildung 17: Beispiel: _automation_wrapper_ – angepasstes TestEnv 32
Abbildung 18: Minimales Beispieltestskript ohne TestEnv 33
Abbildung 19: Test-Handler-Beispiel 35